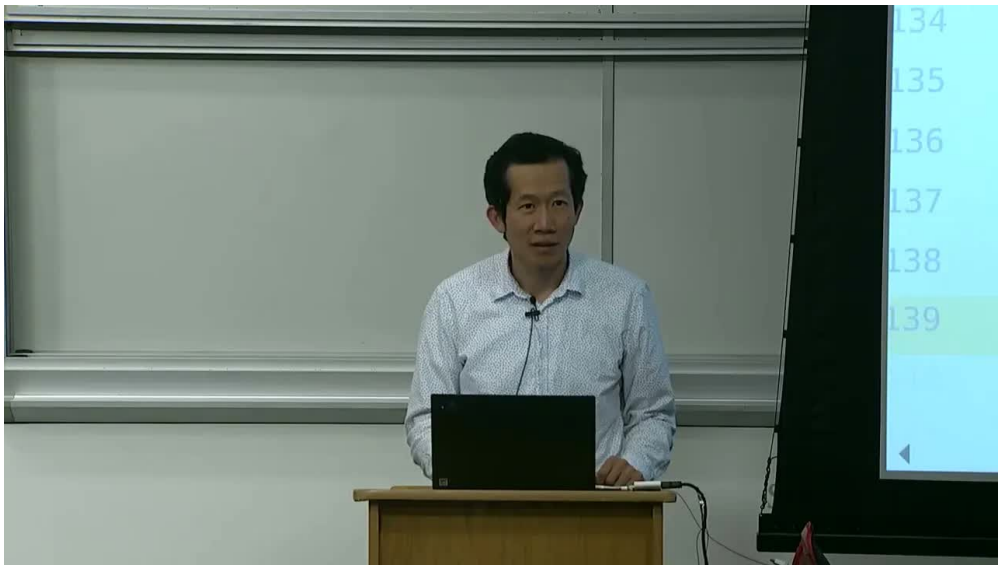
课程笔记

CS336 第 10 讲：推理系统

Codex 基于 Stanford CS336 2025 第 10 讲视频、字幕与课程材料整理

2026 年 4 月 9 日



视频作者/频道：Stanford Online
发布日期：2025 年 05 月 19 日
视频时长：1:22:51
视频链接：`https://www.youtube.com/watch?v=fcgPYo3OtV0`

目录

1 推理系统：为什么“模型训完了”才刚刚开始 **2**

1.1 推理是产品、评测和 RL 的共同底座 2

1.2 三个指标：TTFT、latency 和 throughput 2

1.3 本章小结 2

2 为什么推理会变成 memory-bound：Prefill 与 Decode 的分裂 **3**

2.1 训练和推理最大的差别是并行方式 3

2.2 课程为什么要从矩阵乘重新算一遍 arithmetic intensity 3

2.3 为什么 MLP 和 attention 在 decode 阶段都难受 4

2.4 MLP 强度近似是 BT，attention 在 generation 里却基本卡死在常数级 4

2.5 KV cache 为什么从优化技巧变成中心对象 4

2.6 课程为什么还要拿 Llama 2 13B on H100 算一遍账 4

2.7 本章小结 5

3 先减小 KV cache：GQA、MLA、CLA 与局部注意力 **5**

3.1 GQA：不是所有 query 都需要独立的 K/V 头 5

3.2 MLA 和 CLA：进一步压缩和共享缓存 6

3.3 局部注意力和状态空间模型：彻底改变历史信息的表示方式 6

3.4 本章小结 6

4 再减小数字成本：量化、剪枝与蒸馏 **6**

4.1 推理是 memory-bound，所以低比特很自然 6

4.2 剪枝和蒸馏是在“去掉贵部分之后再修好模型” 7

4.3 本章小结 7

5 不改答案，只重写执行：Speculative Decoding 与动态调度 **7**

5.1 Speculative decoding 的关键不是猜，而是便宜地检查 7

5.2 真实线上流量不是整齐矩阵，而是破破烂烂的 ragged array 8

5.3 continuous batching 和 selective batching 解决的是两个不同层面 8

5.4 PagedAttention：把 KV cache 当成操作系统页来管理 9

5.5 本章小结 9

6 总结与延伸 **10**

1 推理系统：为什么“模型训完了”才刚刚开始

1.1 推理是产品、评测和 RL 的共同底座

第 10 讲首先把推理的重要性讲得非常直接。语言模型的推理不只是聊天机器人上线之后才会遇到的问题，它同时出现在模型评测、测试时计算、强化学习采样、批量离线处理和 API 服务之中。训练也许是一次性的巨额投入，但推理是反复发生、持续付费、直接决定产品单位经济性的过程。

课程用两个现实信号来强调这一点：一方面，前沿模型提供商每天都在消耗极大的推理 token；另一方面，面向程序员和办公场景的应用一旦形成高频流量，推理成本就会迅速压过一次性的训练成本。也正因此，推理优化不是“模型做完后的收尾工作”，而是整个语言模型产业的主战场之一。

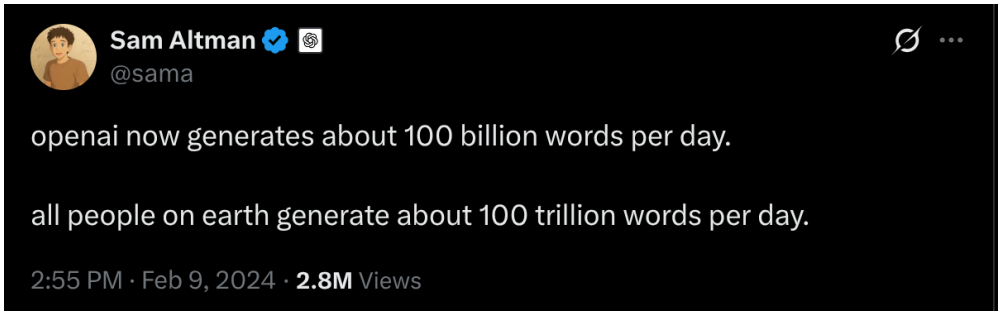


图 1: 课程用真实产品流量提醒学生：推理不是小尾巴，而是持续发生的主成本。

1.2 三个指标：TTFT、latency 和 throughput

课程把推理目标拆成三个互相拉扯的系统指标：

- TTFT（time-to-first-token）：用户多久能看到第一个 token；
- latency（seconds/token）：流式生成时每个 token 出现得有多快；
- throughput（tokens/second）：系统在整体上能处理多少 token。

这三者没有统一最优解。小 batch 往往更有利于 TTFT 和单请求体验，大 batch 则更利于吞吐；一旦系统面对的是动态到达、长度不齐、前缀共享的真实流量，这种冲突会更加明显。因此第 10 讲不是在教某一种固定 recipe，而是在建立一套权衡语言：**不同应用在优化不同目标**。

第 10 讲的核心转向

训练阶段问的是“怎样把已知 token 高吞吐地过一遍模型”，推理阶段问的是“怎样在动态、稀疏、串行的请求流中尽量降低等待和成本”。这两个问题长得像，但不是同一个系统问题。

1.3 本章小结

课程先把推理的应用面和评价指标立起来。只有当你明确自己更在意 TTFT、流式延迟还是总体吞吐时，后面那些看似五花八门的优化技巧才会有清晰的评价标准。

2 为什么推理会变成 memory-bound：Prefill 与 Decode 的分裂

2.1 训练和推理最大的差别是并行方式

课程最重要的一刀，是把推理拆成 prefill 和 decode 两个阶段。prefill 面对的是完整 prompt，可以像训练那样对整段输入并行做矩阵乘，因此更容易逼近 compute-bound；decode 面对的是“已经生成到这里了，下一 token 是什么”，每一步都必须顺序展开，于是并行性迅速下降。

课程用算术强度的语言解释这种差异。对一个典型线性层，粗略地说：

$$\text{FLOPs} \approx 6(BT)\big(\text{num_params} + O(T)\big),$$

其中 B 是 batch，T 是当前并行处理的 token 数。prefill 阶段的 T 较大，因此矩阵乘容易摊薄权重读取成本；decode 阶段通常 $T = 1$，模型要为每个新 token 反复读取大量参数，计算远赶不上内存搬运。

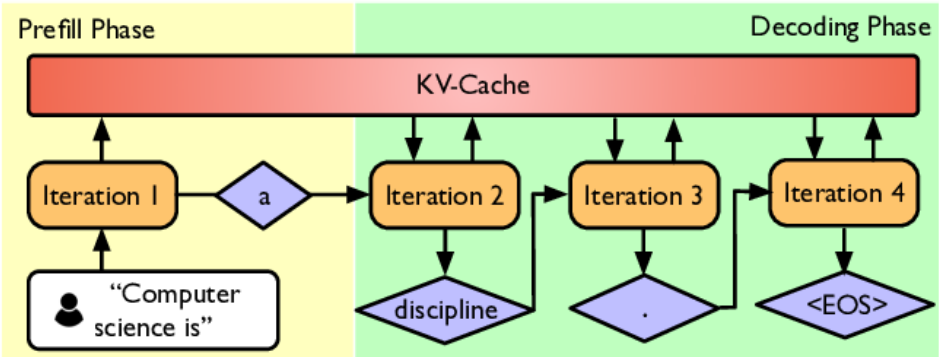


图 2: 课程明确区分 prefill 和 decode：前者像并行编码，后者像带 KV cache 的逐步递推。

2.2 课程为什么要从矩阵乘重新算一遍 arithmetic intensity

这讲最“硬核”也最有价值的一段，其实是重新做算术强度核算。课程先从最普通的矩阵乘 $X(B\times D)\cdot W(D\times F)$ 出发：要读入输入矩阵和权重矩阵，做乘加，再把输出写回 HBM。对应的 FLOPs 约为

$$2BDF,$$

而读写字节数约为

$$2BD + 2DF + 2BF,$$

如果把 bf16 看成每个数 2 字节，那么 arithmetic intensity 大致是

$$\frac{2BDF}{2BD + 2DF + 2BF}.$$

在 $D, F \gg B$ 的近似下，它会塌缩到和 B 同阶。这就是课程那句“矩阵向量乘本质上很亏”的来源：当 $B = 1$ 时，算术强度几乎只有 1，系统天然偏 memory-bound。

第 10 讲里最值得记住的一个估算

如果一个算子的 arithmetic intensity 明显低于硬件的 ‘FLOPs / memory bandwidth‘，那么即使算力指标看起来很高，它在推理里也更可能被内存拖住，而不是被 ALU 拖住。

2.3 为什么 MLP 和 attention 在 decode 阶段都难受

课程进一步区分了两种瓶颈。MLP 层在 decode 时仍然可以通过增加并发请求数 B 提高一些算术强度，因此它的问题是“难以把 B 做得足够大”；attention 层则更糟，因为每个请求有自己的 KV cache，随着历史长度 S 增长，attention 的读写几乎不从 batch 中受益。

这就是课程那句高度压缩的结论来源：**prefill 往往 compute-limited，generation 往往 memory-limited**。对推理系统来说，瓶颈不再是“算不动”，而是“每一步都要重新把很多东西从内存里搬出来”。

2.4 MLP 强度近似是 BT，attention 在 generation 里却基本卡死在常数级

官方 `lecture_10.py` 在这里给了比口头总结更细的 accounting。对 MLP 层，只看三次主要矩阵乘（`Wup` / `Wgate` / `Wdown`）时，总 FLOPs 约为

$$6BTDF,$$

而在 $D, F \gg BT$ 的近似下，intensity 会接近 BT。这就是为什么 prefill 相对好办：只要 prompt 足够长，或者一次并发请求足够多，MLP 仍有机会把权重读取摊薄。

attention 则不同。若用 FlashAttention 风格只看主要矩阵乘，强度近似会变成

$$\frac{ST}{S+T}.$$

于是：

- prefill 时 $T = S$，强度大约是 $S/2$，可以随着上下文长度上升；
- generation 时 $T = 1$，强度逼近 $S/(S+1) \approx 1$，几乎不会因为 batch 变大而改善。

这就是课程一句非常关键的话背后的数学骨架：**MLP intensity 还会随 batch 增长，但 generation 阶段 attention intensity 基本卡在 1，批处理帮不上忙**。

为什么 decode 比训练更难把硬件吃满

训练看见的是一整块规则的张量，decode 看见的是大量短小、不同步、不断变化的请求状态。硬件本身并没有变差，是工作负载变得更不规则了。

2.5 KV cache 为什么从优化技巧变成中心对象

没有 KV cache 时，每生成一个新 token，都要把全部前缀重新过一遍 attention，复杂度和重复工作都不可接受。有了 KV cache，前缀的 key/value 可以复用，于是推理才真正可做。但这也把系统问题重新表述成了另一个更难的问题：**怎样在显存里存、取、共享、压缩和分页这些缓存**。

课程在这里把推理的中心对象从“模型参数”转成了“参数 + KV cache”。参数决定基础读取成本，KV cache 则决定序列越长时代价如何膨胀。后面几乎所有优化，要么是在减少 KV cache，要么是在更聪明地管理 KV cache。

2.6 课程为什么还要拿 Llama 2 13B on H100 算一遍账

第 10 讲不是只停在“prefill 和 decode 很不一样”这种定性层面，它还拿一个具体模型形状做了粗算。对一个近似的 Transformer，如果参数用 bf16 保存，那么参数存储大致是

$$\text{parameter_bytes} \approx 2 \cdot \text{num_params},$$

而单条序列的 KV cache 大致是

$$\text{kv_cache_bytes} \approx S \cdot (KH) \cdot L \cdot 2 \cdot 2,$$

其中最后两个 2 分别对应 key/value 两份缓存和 bf16 每元素 2 字节。于是总内存占用可粗写为

$$\text{memory} \approx \text{parameter_bytes} + B \cdot \text{kv_cache_bytes}.$$

如果把 latency 粗暴近似成一次 step 里总读写量除以内存带宽，就会得到

$$\text{latency} \approx \frac{\text{memory}}{\text{bandwidth}}, \qquad \text{throughput} \approx \frac{B}{\text{latency}}.$$

课程接着比较 batch size 为 1、64、256 的情况，结论很朴素但极重要：更大 batch 往往能提高总体 throughput，但会恶化单请求 latency，而且随着 KV cache 一起膨胀，最终还会直接撞上显存上限。

为什么 TTFT 和 throughput 常常互相打架

prefill 更在意 TTFT，因此希望 batch 小一点；generation 更在意吞吐，因此希望 batch 大一点。对线上系统来说，这两个阶段最优策略甚至可能不是同一个 batch policy。

2.7 本章小结

第 10 讲真正建立的是一个负载模型：prefill 更像批处理编码，decode 更像带状态的逐步递推；前者更受算力限制，后者更受内存限制。只有接受这个分裂，后面的技巧才会显得合理。

3 先减小 KV cache：GQA、MLA、CLA 与局部注意力

3.1 GQA：不是所有 query 都需要独立的 K/V 头

课程先讲 grouped-query attention。标准 MHA 里，每个 query head 都有自己的 key/value 头；MQA 则把所有 query 头压到同一组 K/V 上。GQA 处在二者之间，让多个 query head 共享一组 K/V，从而在准确率和缓存大小之间找折中。

它为什么有效？因为推理时真正贵的是缓存 K/V，而不是 query 本身。减少 K/V 头数，就等于直接减少 KV cache 的体积，也就能在同样显存下容纳更大的 batch 或更长上下文。课程同时强调，GQA 不是越激进越好，组数太少时性能会明显下降。

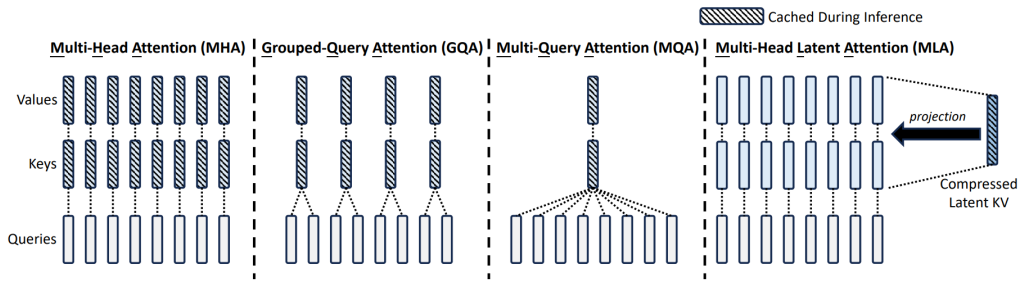


图 3: 课程把 MHA、GQA、MQA 和 MLA 放在同一张图里比较，本质都是在压缩推理时要缓存的 K/V 表示。

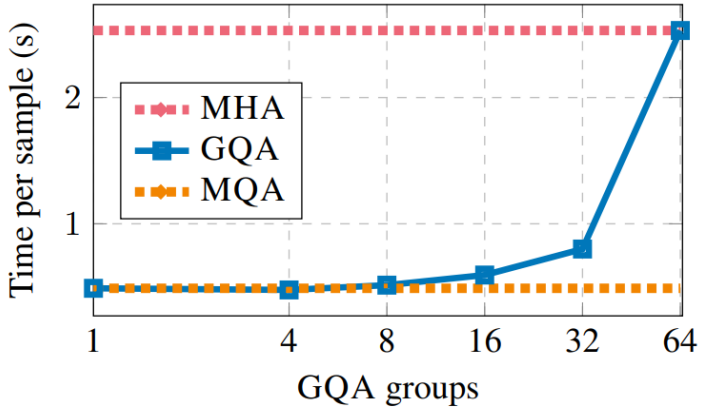


图 4: GQA 通过减少 K/V 头数改善推理时间，但组数选得太极端也会让代价重新上升。

3.2 MLA 和 CLA：进一步压缩和共享缓存

课程接着讲 multi-head latent attention。它不只是减少 K/V 头数，而是把高维的 key/value 先投影到更小的 latent 空间，再缓存这一份压缩后的表示。DeepSeek v2 的例子说明，MLA 可以把缓存维度压得很小，同时仍保留较强性能。

cross-layer attention（CLA）则是另一条线：不是在头之间共享，而是在层之间共享一部分 K/V。直观上，它利用的是“不同层不一定都需要独立维护整份长历史缓存”这件事。GQA、MLA、CLA 三者虽然细节不同，但课程想传达的是统一思路：**推理瓶颈在缓存，于是结构设计就应该围绕缓存压缩来展开**。

3.3 局部注意力和状态空间模型：彻底改变历史信息的表示方式

如果继续沿着“缓存太大”这条线往前走，就会走到局部注意力、线性注意力和状态空间模型。局部注意力的思想是：真正相关的信息通常更靠近当前位置，因此没必要每一层都看完整历史。课程同时提醒，这种做法会损失一部分全局依赖能力，所以往往要和少量全局层混用。

更激进的路线则是状态空间模型。它试图用常数或亚线性的状态表示取代随着序列长度线性增长的 KV cache。课程在这里没有要求学生掌握 S4、Mamba、Jamba 的全部细节，而是要你看到同一个方向：**如果 Transformer 天生不适合推理，那就需要新架构而不是只做局部补丁**。

3.4 本章小结

GQA、MLA、CLA、局部注意力和状态空间模型看起来风格各异，但它们在系统意义上做的是同一件事：减少或重写推理阶段必须保留的历史状态。

4 再减小数字成本：量化、剪枝与蒸馏

4.1 推理是 memory-bound，所以低比特很自然

课程在讨论量化时一直重复一条主线：如果推理主要受内存搬运限制，那么把参数和缓存变小就几乎一定有帮助。bf16、fp8、int8、int4 这些格式不是为了数学优雅，而是为了少搬几倍字节。于是推理优化就从“如何少算一点”扩展成“如何让每一步少读一点”。

但课程也明确区分了两类做法。第一类是 post-training quantization，在不重新大规模训练的前提下，用校准数据估计各层缩放因子；第二类是 quantization-aware training，直接在训练里把低比特误差纳入建模。前者便宜，后者通常更稳，但更贵。

4.2 剪枝和蒸馏是在“去掉贵部分之后再修好模型”

量化改的是数值精度，剪枝和蒸馏改的是模型结构本身。课程把这类方法理解成：先识别真正重要的层、头、隐藏维，再删掉代价高但相对不重要的部分，最后用蒸馏把性能修回来。它们和 GQA/MLA 不同，不是直接为 KV cache 设计，而是在更宏观层面重写“这个模型到底需要多大”。

速度提升从来不是白拿的

GQA、MLA、量化、剪枝、蒸馏都属于 lossy shortcut。它们通过修改模型本身换取速度和成本，因此必须跟着做准确率或能力验证；不能因为系统指标好看，就默认答案质量没有变化。

4.3 本章小结

量化、剪枝和蒸馏把“推理优化”从运行时技巧推进到模型本体改造。只要推理仍受内存限制，减少比特数和减少模型体积就始终会是强有力的方向。

5 不改答案，只重写执行：Speculative Decoding 与动态调度

5.1 Speculative decoding 的关键不是猜，而是便宜地检查

课程把 speculative sampling 讲得很清楚：用一个便宜 draft model 先提出若干 token，再让目标模型并行检查这些提案。如果提案与目标模型分布足够一致，就一次接受多个 token；不一致时再做修正。它利用的是一个核心不对称性：**检查一串候选 token 通常比从零开始逐个生成更便宜**。

更重要的是，课程强调这种方法在数学上可以做到对目标模型的 exact sampling，而不是“差不多像”。这点非常关键，因为它把 speculative decoding 和普通近似启发式分开了。它不是用错误答案换速度，而是在不改变采样分布的前提下重排执行顺序。

Algorithm 2 Speculative Sampling (SpS) with Auto-Regressive Target and Draft Models

Given lookahead K and minimum target sequence length T.
Given auto-regressive target model $q(.|.)$, and auto-regressive draft model $p(.|.)$, initial prompt sequence $x_0, \ldots, x_t$.
Initialise $n \leftarrow t$.
while $n < T$ **do**
  for $t = 1 : K$ **do**
    Sample draft auto-regressively $\tilde{x}_t \sim p(x|, x_1, \ldots, x_n, \tilde{x}_1, \ldots, \tilde{x}_{t-1})$
  end for
  In parallel, compute $K + 1$ sets of logits from drafts $\tilde{x}_1, \ldots, \tilde{x}_K$:

$$q(x|, x_1, \ldots, x_n),\ q(x|, x_1, \ldots, x_n, \tilde{x}_1),\ \ldots,\ q(x|, x_1, \ldots, x_n, \tilde{x}_1, \ldots, \tilde{x}_K)$$

  for $t = 1 : K$ **do**
    Sample $r \sim U[0, 1]$ from a uniform distribution.
    if $r < \min\left(1, \frac{q(x|x_1,\ldots,x_{n+t-1})}{p(x|x_1,\ldots,x_{n+t-1})}\right)$, **then**
      Set $x_{n+t} \leftarrow \tilde{x}_t$ and $n \leftarrow n + 1$.
    else
      sample $x_{n+t} \sim (q(x|x_1, \ldots, x_{n+t-1}) - p(x|x_1, \ldots, x_{n+t-1}))_+$ and exit for loop.
    end if
  end for
  If all tokens $x_{n+1}, \ldots, x_{n+K}$ are accepted, sample extra token $x_{n+K+1} \sim q(x|, x_1, \ldots, x_n, x_{n+K})$ and set $n \leftarrow n + 1$.
end while

图 5: speculative decoding 的本质是：让小模型提案，让大模型并行验证并修正。

5.2 真实线上流量不是整齐矩阵，而是破破烂烂的 ragged array

课程后半段转向服务系统的真正难点。训练时你有整齐的 $B \times S$ 张量；线上推理时，请求在不同时间到达、长度不同、可能共享前缀，还会在不同时间结束。于是系统看到的不是规整批，而是一个不断变化的 ragged workload。

这时就需要 iteration-level scheduling 和 selective batching。前者让系统在每个 decode step 重新组织 batch，把新请求插进来；后者则区分 attention 和非 attention 部分，对形状要求严格的部分按序列处理，对可以拼接的部分合并计算。课程借这里强调：**推理服务不是单个模型调用，而是一个调度问题**。

5.3 continuous batching 和 selective batching 解决的是两个不同层面

官方 lecture script 在这里拆得比当前笔记更细。continuous batching 解决的是“请求什么时候入队、什么时候和别的请求一起解码”的问题；selective batching 解决的是“即使序列长度不同，哪些算子还能合并跑”的问题。两者经常被一起提，但不是一个概念。

更具体地说：

- attention 部分通常仍要按序列边界处理，因为每条序列的有效长度不同；
- 非 attention 部分，例如许多逐 token 的线性层和 MLP，可以把不同序列的 token 拼成一条更长的 batch 来统一跑。

这就是为什么课程会说线上推理看到的是 ragged array：真正难的不是“有没有 batch”，而是“不同部位能否以不同方式 batch”。

5.4 PagedAttention：把 KV cache 当成操作系统页来管理

vLLM 的 PagedAttention 是课程给出的代表性例子。过去的做法常常给每个请求预留一整段连续 KV cache，导致大量内部和外部碎片。PagedAttention 则把缓存切成块，让一个序列的逻辑地址映射到一组非连续物理块上。这样不同长度请求可以更高效共存，共享前缀也更容易做到 copy-on-write。

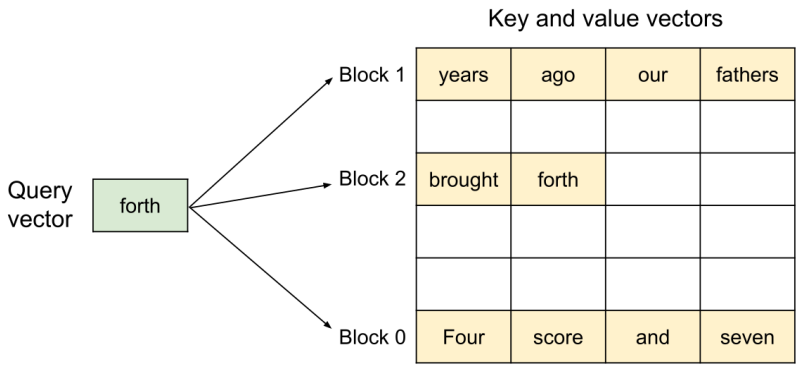


图 6: PagedAttention 的核心思想来自操作系统：把 KV cache 分块，让逻辑顺序和物理存储解耦。

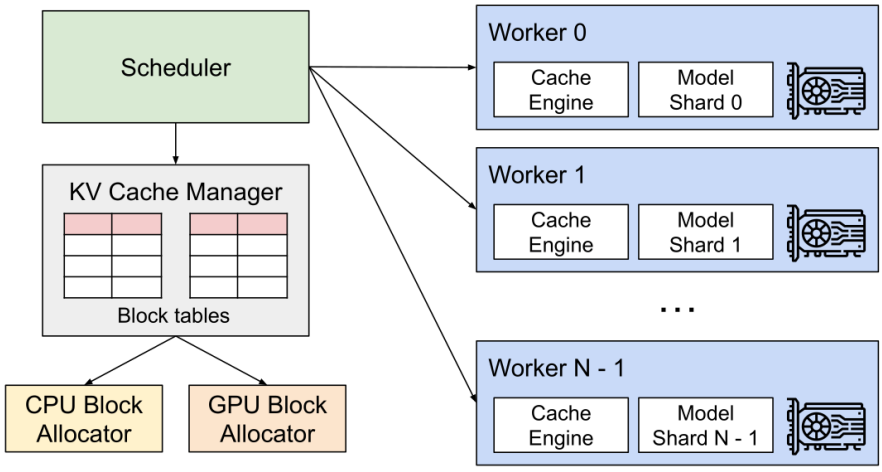


图 7: vLLM 的系统结构图把 scheduler、KV cache manager、CPU/GPU block allocator 和 worker 分开管理。

课程在这里特别漂亮地把系统课和模型课连了起来。到了线上服务阶段，瓶颈已经不只是 attention 数学本身，而是显存碎片、请求调度、前缀共享和 kernel launch overhead 这些纯工程问题。

5.5 本章小结

speculative decoding 和 PagedAttention 代表了两种“lossless shortcut”。前者利用便宜提案和昂贵检查之间的不对称，后者利用操作系统式的分页和共享来提高内存利用率。二者的共同点是：答案不变，但执行方式完全重写。

6 总结与延伸

第 10 讲最值得带走的是一个统一的推理视角。

第一，**推理不是训练的简化版，而是另一种工作负载**。它同时服务于产品、评测和 RL，因此不能被当成后处理问题。

第二，**prefill 和 decode 必须分开理解**。前者更接近 compute-bound，后者更接近 memory-bound，而所有系统技巧基本都是在对付 decode。

第三，**KV cache 是推理系统的中心对象**。 GQA、MLA、CLA、局部注意力、状态空间模型，本质上都在减少或重写需要缓存的历史表示。

第四，**推理优化分成 lossy 和 lossless 两大类**。量化、剪枝、蒸馏、结构改造会修改模型本身；speculative decoding、continuous batching、PagedAttention 则主要重写执行流程。

第五，**线上推理是调度问题，也是内存管理问题**。请求动态到达、前缀共享和长度不齐，让真实系统远比论文里的单样本 latency 分析复杂。

我对第 10 讲的压缩提醒

如果只把这讲记成“推理有 KV cache、可以做量化和 speculative decoding”，那还是太浅。真正重要的是：**推理系统的瓶颈来自 memory-bound decode 和动态工作负载，因此优化必须同时动模型结构、数值表示和执行调度**。

从课程主线看，第 10 讲把系统视角推进到了产品级推理；第 11 讲则会回到 scaling recipe，讨论如何把这种系统约束重新折回训练阶段的设计和超参数外推中去。